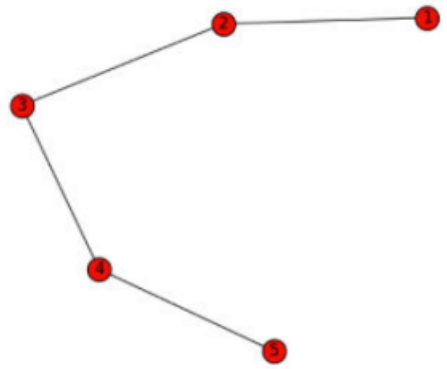
Algoritmizace – Grafy, prohledávání stavového prostoru, řazení

Grafy

Grafy jsou způsobem reprezentace vztahů a propojení mezi určitými objekty / prvky. Objektům jsou přiřazeny vrcholy (téže zvané uzly) a při zobrazení vztahu propojíme dva vrcholy přímkou – hranou.

Cesta

Cesta je posloupnost vrcholů a hran, kde existuje pouze jeden způsob, jak se dostat jednoho vrcholu do jiného.



Ohodnocený graf

Ohodnocený graf nám říká, jaká je vzdálenost mezi určitými vrcholy, případně délku hrany. Může také ale spoužit k jednoduchému přiřazení čísel ke hranám.

Úplný graf

Jedná se o graf, ve kterém je každý vrchol spojen se všemi ostatními vrcholy hranou.

Cyklický a necyklický

Cyklický graf je graf, ve kterém můžeme chodit dokola nekonečně skrze všechny body a necyklický

graf je takový, ve kterém se můžeme zaseknout a nejít skrze všechny body jednou cestou bez vracení.

Strom

Stromem se nazývají souvislé grafy, které neobsahují kružnice.

Strom je tedy minimální souvislý graf na daných vrcholech.

Stromy se v informatice velmi často vyskytují, protože reprezentují základní koncept, kterým je hierarchie.

Se stromy se setkáme všude tam, kde je potřeba rychle vyhledávat, reprezentovat strukturovaná data nebo rozhodovat.

Strom je i přirozeným modelem rekurze.

- kořen (root) – horní prvek
- vnitřní uzel (inner node) = uzel, který není kořenem, ani listem
- list (leaf node, external node) = uzel, který nemá žádné potomky
- rodič (parent node) = uzel, který přímo předchází daný uzel na cestě od listu ke kořeni
- potomek (child node) = uzel, který přímo následuje za daným uzlem na cestě od kořene k listu
- sourozenec (sibling) = jako sourozenci se označují uzly se stejným rodičem
- předek (ancestor node, predecessor node) = uzel, který leží před daným uzlem na cestě ke kořeni (nejbližší předek je rodič)
- následník (successor node) = uzel, který leží za daným uzlem na cestě od kořeni k libovolnému listu (nejbližší následník je potomek)
- hloubka (depth) = hloubka stromu je délka nejdelší cesty od kořene k listu, přičemž prázdný strom má definovánu hloubku jako -1
- úroveň (level) = většinou se používá ve významu množiny uzlů, které se nachází ve stejné vzdálenosti od kořene, počítáno dle počtu uzlů

Používají se jako datové struktury, ty ale používají binární strom (max 2 potomky na uzel) a

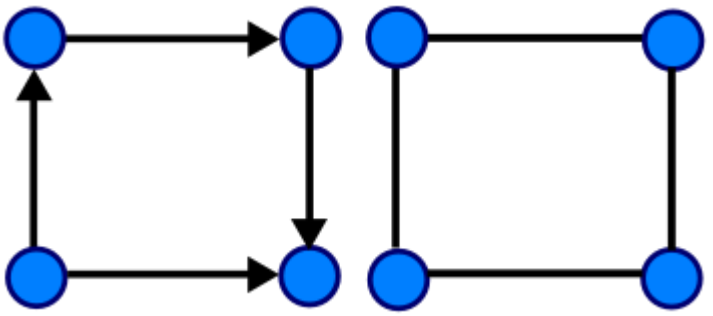
často upravený, Red-black, halda,… jeho výhoda je, že při vyhledání neprochází všechny

prvky grafu.

Dělení grafů

Orientovaný a neorientovaný graf

U orientovaných grafů se jedná o uspořádané dvojice, zatímco u neorientovaných se jedná pouze o množinu prvků. Orientovaný graf poznáme podle toho, že místo přímky jako hrany využijeme šipku, čímž definujeme směr. Směr nám definuje, kudy se můžeme po hraně pohybovat.

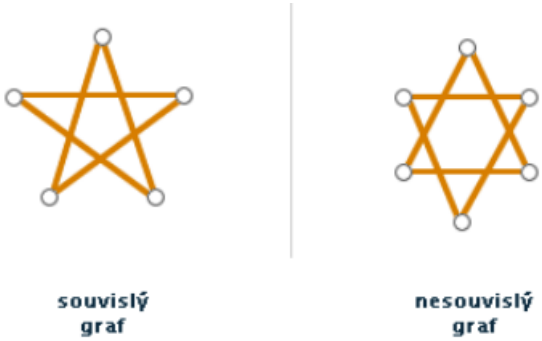


Spojitý a nespojitý (souvislý a nesouvislý)

Spojitý, z každého bodu vede alespoň jedna cesta do jiného a celkově jsou všechny body spojeny.

Nespojitý, z bodu se nedá žádnou možností dostat do jiného

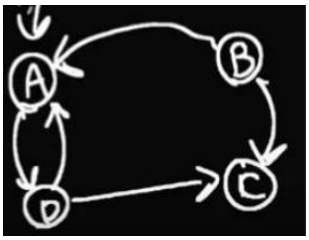
Spojité se lépe ukládají do počítače.



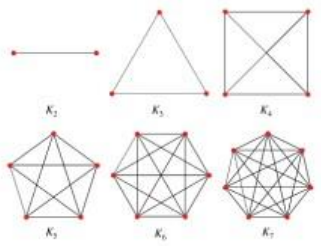
Matice sousednosti

Tato matice uvádí pro každý vrchol spojené se sousedy čtvercovou matici kde každý soused nám vytvoří diagonálu grafem

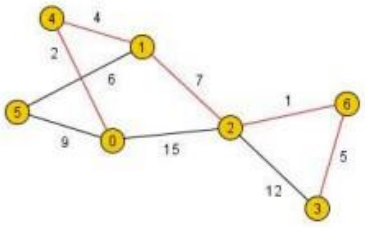
Necyklický graf s neorientovanou stranou AB



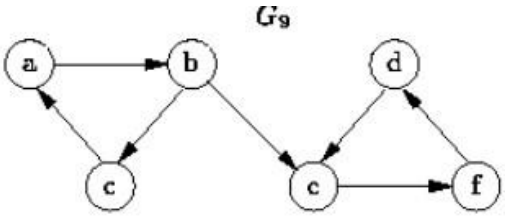
Úplný graf



Ohodnocený graf



Orientovaný graf s okruhem A-B-C a C-D-F



Prohledávání stavového prostoru

Časová složitost

Minimální, maximální a průměrný čas potřebný k prohledání danou metodou.

Prostorová složitost

Minimální, maximální a průměrné množství operační paměti potřebné k prohledání. Tam se počítá kolikrát je prvek uložen, třeba proměnná max do sebe uloží nějaké číslo a metoda zabírá počet prvků + 1, ale to je zanedbatelné, horší to je u vytvoření pomocných kolekcí.

Kvalita získaných výsledků

Určuje, jestli metoda prohledání dá výsledek pokaždé

Prohledávání

Prohledávání do hloubky (Depth-First Search) – Vybere se směr a tím se jde až na konec.

Směr se pomatuje, a proto se algoritmus nezacyklí. Je implementován pomocí Stacku

Výhody

Nevyžaduje tak velkou paměť jako BFS. Pokud se prošla levá větev a v ní nic nebylo, tak si stačí pamatovat jen kořen té větve.

Pokud víme, že je řešení ve velké hloubce, tak máme šanci najít řešení mnohem rychleji než BFS.

Prohledávání do šířky (Breath-First Seach)

Prohledá všechny sousedy od startovního prvku, poté sousedy sousedů atd. až projde celou komponentu souvislosti. Je implementován pomocí Queue.

Výhody

Najde nejkratší cestu - za předpokladu, že je graf neohodnocený (vzdálenost mezi větvemi je stejná)

Umí hledat řešení i v nekonečných či opravdu velkých grafech.

Je-li více řešení a některá jsou blíž, některá dál – najde nejbližší.

Djikstrův algoritmus

Používá se pro nalezení nejkratší cesty grafem.

Je konečný (pro jakýkoliv konečný vstup algoritmus skončí), protože v každém průchodu cyklu se množiny navštívených uzlů přidá právě jeden uzel, průchodů cyklem je tedy nejvýše tolik, kolik má graf vrcholů.

Funguje na hranově kladně ohodnoceným grafem.

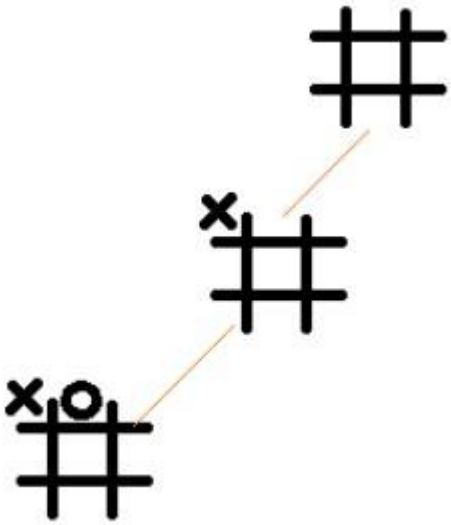
Stavový prostor

Stavový prostor je soubor všech stavů, které mohou v určitém problému nastat. V Javě se dá uložit do různých datových struktur, výhodná datová struktura je například strom.

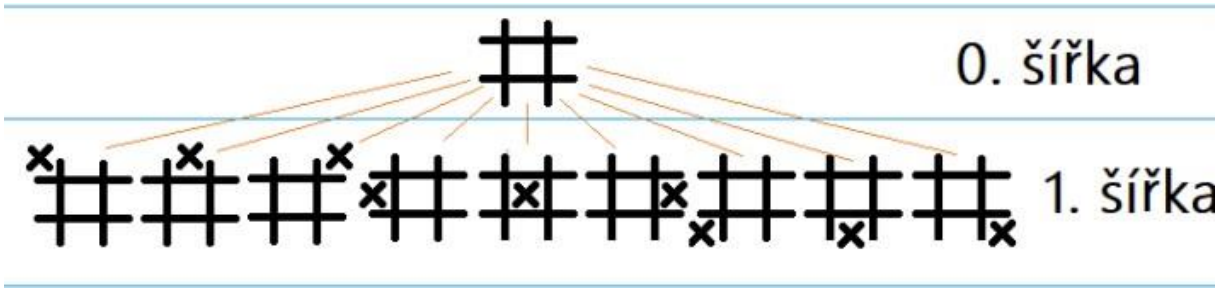
Prohledávání stavového prostoru je určitá akce, ve kterém se snažíme najít určitý stav, který je nejlepší/ nejvýhodnější / výsledkově pro nás dobrý. Prohledávání stavového prostoru jde do dvou směrů – do šířky, nebo do hloubky.

Dobrý příklad mohou být piškvorky, kde v případě, že si jsme jisti, že začíná křížek, je jako první stav prázdná mřížka

Při prohledávání do hloubky se vytváří nový stav, který okamžitě generuje další. Na obrázku vidíme dva první kroky prohledávání algoritmem do hloubky. Příklad hledání do hloubky



Při hledání do šířky vygenerujeme nejdříve jednu vrstvu a poté všechny stavy v této vrstvy vygenerují další.



Vzhledem k tomu, že je stavový prostor množina všech možných výsledků, je potřeba k jeho kompletnímu projetí hrubá síla – **bruteforce algoritmus**.

Třídící algoritmy

Pokud se třídí objekty (vlastní), tak musí mít implementované rozhraní IComparable a metodu compareTo.

```
class Nevim : IComparable<Nevim>
{
    public int Cislo;
    Počet odkazů: 5
    public Nevim(int cislo)
    {
        this.Cislo = cislo;
    }
    Počet odkazů: 0
    public int CompareTo(Nevim other)
    {
        Nevim d = (Nevim)other;
        return this.Cislo.CompareTo(d.Cislo);
    }
}
```

Nebo mít vytvořený IComparer pro tyto objekty a ten dát algoritmu.

```
class NevimComparer : IComparer<Nevim>
{
    Počet odkazů: 0
    public int Compare(Nevim x, Nevim y)
    {
        if (x.Cislo > y.Cislo)
            return 1;
        if (x.Cislo < y.Cislo)
            return -1;
        else
            return 0;
    }
}
```

Májí za úkol seřadit nějakou skupinu (kolekci) prvků.

Třídicí metoda v C# .Sort()

Pokud má kolekce míň jak 16 prvků nebo rovno, tak se použije Insertion Sort

Pokud počet oddílů přesáhne 2*Log(n) tak se použije Heap Sort

Jinak se používá Quick Sort,

Ale pokud se použije LINQ metoda třeba OrderBy(), tak je vřdy použit Quick Sor

Řazení je operace, během kterého srovnáme určitou datovou sturkturu plnou určitých dat dle potřeb.

Využívají se k tomu různé typy řazení, každé má svojí rychlost a každý je vhodný pro určitý počet dat.

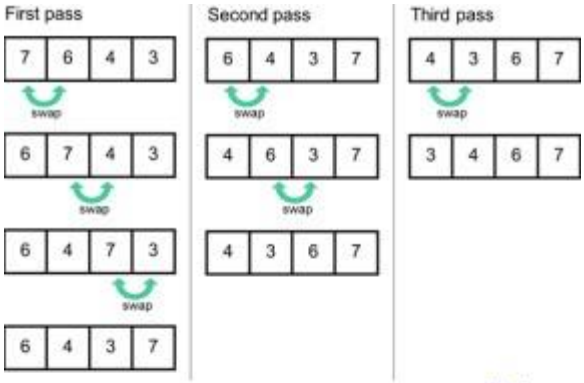
Je hodně sortů, třeba Radix, Selection, Stalin sort, Heap sort, ale nejznámější jsou tyto.

Bubble Sort

Bubble sort funguje jednoduše, vezme první dva prvky a na základě velikosti je vymění. Poté vezme následující prvek a udělá to samé.

Bubble Sort potřebuje celý jeden cyklus k tomu, aby mohl uznat, jestli je pole setříděné nebo ne.

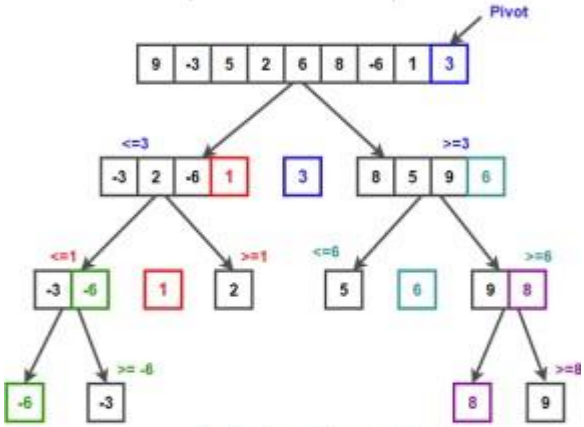
Časová náročnost: Kvadratická, n2



Merge Sort

„Rozděl a panuj“. Tento algoritmus vezme nesetříděné pole a dělí je na poloviny tak dlouho, dokud nejsou pouze dvojice. Následně dvojici seřadí a poté všechny dvojice spolu s další dvojicí spojuje tak dlouho, dokud nemá celé pole.

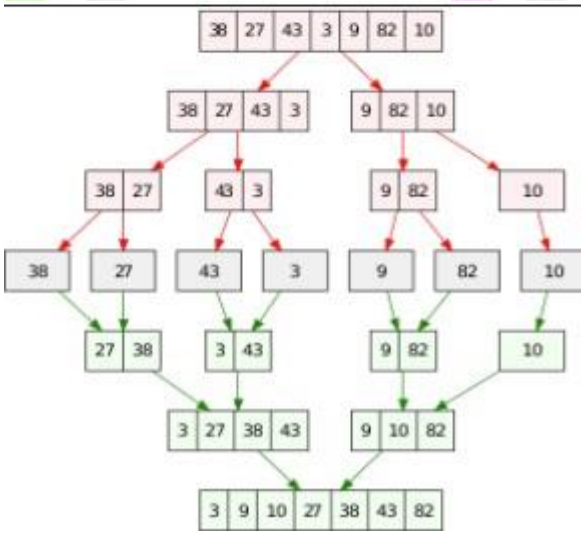
Časová náročnost: n log n



Quick Sort

Quick sort spočívá ve vybrání pivota. Podle pivota následně rozřadíme na prvky, které jsou menší a na prvky, které jsou větší. Ony dvě skupiny zase budou mít svého pivota, který podskupiun rozdělí na další podpodskupiny a takhle do té doby, dokud není seřazeno

Časová průměrná náročnost: n log n



Selection Sort

Jednoduchý algoritmus

Projde se celá kolekce a najde se nejmenší prvek, ten si prohodí místo s prvním prvkem v kolekci.

Poté se projde kolekce znova bez prvního prvku a najde se zase nejmenší prvek a prohodí si místo s druhým. Poté se projde kolekce bez 2 na začátku.

Tohle se opakuje až do konce pole.

Nejlepší čas: O(n^2)

Průměrný čas: O(n^2)

Nejhorší čas: O(n^2)

Paměť: 1, nepoužívá místo navíc, ale pokud třídí LinkedList tak místo je O(n)

Stabilní: ne

Insertion Sort

Rozdělí kolekci na 2 části: setříděnou a nesetříděnou

Do setříděné spadá na začátku pouze první prvek.

Vybere se první prvek nesetříděné části (druhý v celé kolekci, když setříděná má jeden)

Porovná se s prvkem v setříděné části, ta se zvětší o 1 místo, pokud je nový prvek větší jak první (co tam už byl) tak půjde za něj, pokud ne, tak jde před něj. Poté se vezme zase první prvek z nesetříděné části (teď 3. V celé kolekci) a porovná se s těma v setříděné části. A takhle to jde až v nesetříděné části nezbydou žádné prvky.

Nejlepší čas: O(n)

Průměrný čas: O(n^2)

Nejhorší čas: O(n^2)

Paměť: 1, nepoužívá místo navíc

Stabilní: ano

Heap Sort

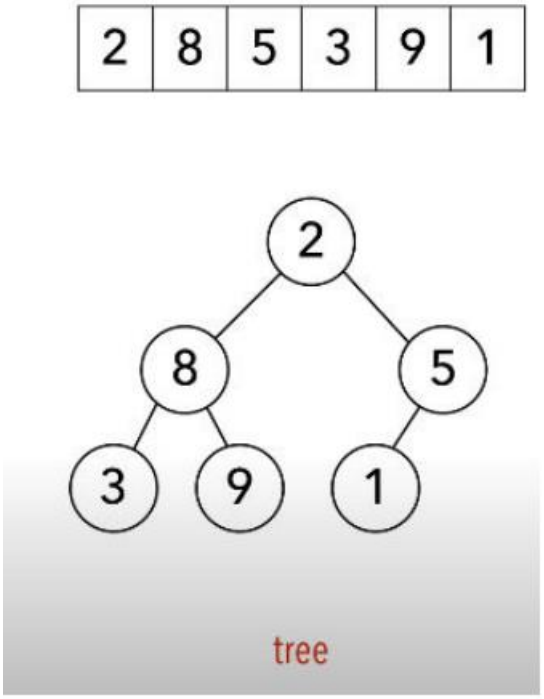
Založen na myšlence Selection Sortu, má za úkol být rychlejší než on.

Ale selection sort je už nejrychlejší jak může bez místa v paměti navíc. Proto se vytvoří z pole naoko strom, konkrétně halda typu max-heap (největší nahoře).

Halda je druh binárního stromu s pravidly.

- Je to plný binárí strom, Všechny levely haldy až na poslední jsou plně obsazeny prvky
- Poslední level je zaplnění zleva, pokud není zaplněno celé
- Pro prvky v haldě platí speciální vlastnost haldy: oba potomci jsou vždy menší nebo rovny rodičovi.

Halda se vytvoří pouze na oko

Pole se představí jako halda, první prvek je kořen a je nahoře, poté se prvky dávají na úrovně zleva

Poté se prvky zpřeházení podle pravidel haldy. Kořen se porovná se svými potomky a pokud je nějaký větší tak si prohodí místo s rodičem (teď kořenem), pokud jsou větší oba, tak si prohodí místo s větším z nich. Poté se přejde na nižší level a to se opakuje plus se kontorluje jestli nově prohozená hodnota není větší než kořen nebo rodič kontorlovaného uzlu